

SMART GRID ENERGY MONITORING AND DISTRIBUTION SYSTEM

CSIZC462: Assignment

by

Satish Y (2024MT13013)
Prabhath K (2025HT66024)

Assignment work carried out at
BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE PILANI (WILP)

Submitted in partial fulfillment of M. Tech. Network Programming
programme

Under the Supervision of
Prof. Vivekananda M R
Instructor In-Charge (I/C) - Network Programming



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE PILANI (WILP)

May, 2026

ACKNOWLEDGEMENT

We sincerely thank our faculty member Mr. Vivekananda M R for his continuous guidance and support throughout the completion of this project. This assignment helped us understand practical concepts of socket programming, inter-process communication, and real-time data transfer in distributed systems.

We also thank BITS Pilani for providing the opportunity to work on a practical network programming project.

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
IPC	Inter Process Communication
TCP	Transmission Control Protocol
IoT	Internet of Things
RAM	Random Access Memory
API	Application Programming Interface

ABSTRACT

The Smart Grid Energy Monitoring and Distribution System is a simple network programming project developed using Python socket programming and IPC concepts. The system simulates sector nodes that continuously generate energy usage data and transmit it to a central monitoring server.

The server monitors the received data in real time and detects abnormal energy usage conditions. The project demonstrates practical implementation of network programming concepts such as TCP socket communication, process coordination, real-time monitoring, and fault detection.

The system is lightweight, easy to understand, and suitable for demonstrating distributed communication concepts in a smart grid environment.

TABLE OF CONTENTS

1. Introduction
2. Requirement Specification
3. Design Methodology
4. Implementation Details
5. Testing
6. Performance Measurement
7. Future Work
8. Conclusion
9. References

1. INTRODUCTION

Smart grid systems are advanced electricity distribution networks that use communication technologies for monitoring and management of energy resources. Traditional grid systems face challenges in real-time monitoring, fault detection, and efficient communication.

This project presents a simple smart grid monitoring prototype using Python programming language. The system uses TCP socket programming and IPC concepts to simulate communication between sector nodes and a central server.

The main objective of this project is to provide:

- Real-time energy monitoring
- Reliable communication
- Centralized monitoring
- Fault detection capability
- Practical implementation of network programming concepts

The system continuously exchanges energy usage information between sector nodes and the server, making the monitoring process efficient and reliable.

2. REQUIREMENT SPECIFICATION

2.1 Hardware Requirements

- Computer system with minimum 4GB RAM
- Intel i3 processor or above
- Stable internet/network connection

2.2 Software Requirements

- Python 3.x
- Linux or Windows Operating System
- VS Code / PyCharm / Terminal

2.3 Functional Requirements

The system should:

- Monitor energy usage continuously
- Transmit data in real time
- Detect abnormal energy conditions
- Support communication between server and sector nodes
- Display warning messages for high energy usage

2.4 Non-Functional Requirements

- Reliable communication
- Low resource usage
- Fast data transfer
- Easy implementation
- Scalability for future enhancements

3. DESIGN METHODOLOGY

3.1 System Architecture

The system consists of the following modules:

1. Sector Node

The sector node generates simulated energy usage values and sends the data to the central server.

2. Central Monitoring Server

The server receives energy data from multiple nodes and monitors the received values continuously.

3. Socket Communication

TCP socket communication is used for reliable data transfer between nodes and servers.

4. IPC Mechanism

IPC concepts are used for communication between local processes and synchronization.

3.2 Workflow

1. Sector nodes generate energy usage values.
2. Data is transmitted to the server using TCP sockets.
3. The server receives and processes the data.
4. If energy usage exceeds the threshold value, a warning message is displayed.
5. Monitoring continues continuously in real time.

4. IMPLEMENTATION DETAILS

4.1 Programming Language Used

The project is implemented using Python programming language because it provides simple libraries for socket communication and threading.

4.2 Concepts Used

- TCP Socket Programming
- Multi-threading
- Inter Process Communication
- Real-time Data Monitoring

4.3 Source Code Description

Server Module

The server continuously listens for incoming client connections and receives energy usage values.

Client Module

The client generates random energy values and sends them to the server periodically.

Fault Detection

If the energy value exceeds the specified threshold, the server displays a warning message.

4.4 Sample Source Code

```
import socket
import threading
import random
import time

HOST = "127.0.0.1"
PORT = 5000

def handle_client(conn, addr):
    print(f"Connected by {addr}")

    while True:
        data = conn.recv(1024).decode()

        if not data:
            break

        value = int(data)
        print(f"Received Energy Usage: {value} MW")

        if value > 80:
            print("Warning: High Energy Consumption Detected")

    conn.close()
```

5. TESTING

5.1 Connection Testing

The communication between server and sector nodes was tested successfully.

5.2 Data Transmission Testing

Continuous transmission of energy usage values was verified.

5.3 Fault Detection Testing

The system successfully detected abnormal energy usage conditions and displayed warning messages.

5.4 Multiple Client Testing

Multiple nodes were able to communicate with the server simultaneously.

6. PERFORMANCE MEASUREMENT

The system performance was evaluated based on communication reliability and data transfer speed.

Observations

- Reliable TCP communication achieved
- Fast transmission of energy values
- Continuous monitoring supported
- Low memory and CPU usage observed

7. FUTURE WORK

The project can be improved further by adding:

- IoT sensor integration
- Cloud-based monitoring dashboard
- AI-based energy prediction
- Database storage support

- Graphical user interface
- Real-time analytics

8. CONCLUSION

The Smart Grid Energy Monitoring and Distribution System successfully demonstrates the practical implementation of network programming concepts using Python.

The project achieved reliable communication between sector nodes and the central server using socket programming and IPC mechanisms. Real-time monitoring and fault detection were implemented successfully.

The project is simple, efficient, and suitable for understanding distributed communication systems and smart grid monitoring concepts.

9. REFERENCES

1. Python Official Documentation
2. Python Socket Programming Documentation
3. Computer Networks by Andrew S. Tanenbaum
4. Smart Grid Research Articles
5. Network Programming Study Material

APPENDIX

Steps to Execute the Program

1. Install Python 3.x
2. Open terminal or command prompt
3. Run the Python file:

```
python smart_grid_system.py
```

4. The server starts automatically
5. Sector nodes begin transmitting energy values
6. Warning messages are displayed for abnormal conditions